

Lab Assignment 6

Topics:

- Array of objects, Constructors in array of objects
- Pointer to objects, this pointer
- Passing and returning objects
- Static and const member function
- Inline function

1. Implement a class ***Library*** that contains **an array of *Book* objects**. The ***Book*** class should have attributes like **title, author, and ISBN**. The ***Library*** class should provide following methods:
 - a. **bool addNewBook(title,author,ISBN)** to add the new book in the book list.
The return type should be Boolean to check whether the book is added or not.
This function should take the argument using pass-by-reference method. The attribute of the class (title, author, and ISBN) and the function arguments of function **addNewBook** should be same. Use the **scope resolution operator** to access the class data members.
 - b. **bool removeBooks(ISBN)** to remove the book using the input parameter ISBN.
The return type should be Boolean to check whether the book is removed or not.
This function also should take the argument using pass-by- reference method.
Define function outside the class using scope resolution operator.
 - c. **displayDetails()** to list the books.

Write a main function to demonstrate adding, removing, and displaying books from the library. Add at least 5 books to demonstrate the adding book, 1 book to remove from the list.

```
#include <iostream>
#include <string>
using namespace std;
```

```
class Book {
public:
    string title, author, ISBN;
};
```

```
class Library {
```

```

Book books[100];

int count;

public:

    Library() { count = 0; }

    bool addNewBook(string &title, string &author, string &ISBN) {
        if(count >= 100) return false;

        books[count].title = title;
        books[count].author = author;
        books[count].ISBN = ISBN;
        count++;
        return true;
    }

    bool removeBooks(string &ISBN);

    void displayDetails() {
        if(count == 0) {
            cout << "No books available\n";
            return;
        }
        for(int i = 0; i < count; i++) {
            cout << books[i].title << " | "
                << books[i].author << " | "
                << books[i].ISBN << endl;
        }
    }
};

bool Library::removeBooks(string &ISBN) {
    for(int i = 0; i < count; i++) {
        if(books[i].ISBN == ISBN) {
            for(int j = i; j < count - 1; j++) {

```

```

        books[j] = books[j + 1];
    }
    count--;
    return true;
}
}
return false;
}

```

```

int main() {
    Library lib;
    int choice;
    string title, author, ISBN;

    do {
        cout << "1.Add 2.Remove 3.Display 4.Exit\n";
        cin >> choice;

        switch(choice) {
            case 1:
                cout << "Enter title, author, ISBN:\n";
                cin >> title >> author >> ISBN;
                if(lib.addNewBook(title, author, ISBN))
                    cout << "Book added\n";
                else
                    cout << "Failed\n";
                break;

            case 2:
                cout << "Enter ISBN to remove:\n";
                cin >> ISBN;
                if(lib.removeBooks(ISBN))
                    cout << "Book removed\n";
                else
                    cout << "Not found\n";

```

```
break;
```

```
case 3:
```

```
lib.displayDetails();
```

```
break;
```

```
}
```

```
} while(choice != 4);
```

```
return 0;
```

```
}
```

```
input
1.Add 2.Remove 3.Display 4.Exit
1
Enter title, author, ISBN:
1984 George 5749235
Book added
1.Add 2.Remove 3.Display 4.Exit
1
Enter title, author, ISBN:
Canthurtme davidgoggins 284433
Book added
1.Add 2.Remove 3.Display 4.Exit
1
Enter title, author, ISBN:
arhenius acrobat 3958443
Book added
1.Add 2.Remove 3.Display 4.Exit
1
Enter title, author, ISBN:
isbt delhi 1947875
Book added
1.Add 2.Remove 3.Display 4.Exit
1
< Enter title, author, ISBN:
atuuka jsk 11111
Book added
1.Add 2.Remove 3.Display 4.Exit
2
Enter ISBN to remove:
1947875
Book removed
1.Add 2.Remove 3.Display 4.Exit
3
1984 | George | 5749235
Canthurtme | davidgoggins | 284433
arhenius | acrobat | 3958443
atuuka | jsk | 11111
1.Add 2.Remove 3.Display 4.Exit
[]
```

2. Implement the question 1 using constructor. Do the following modification:
 - a. Add default constructor, parameterized constructor, and copy constructor to add the details of the book using the same parameters, i.e., **title, author, and ISBN**.

- b. Use the “this” pointer instead of scope resolution operator to access the data members of the class inside the constructor and function definition.
- c. Create function **bool removeBooks(ISBN)** same as the question 1.
- d. **displayDetails()** same as question 1.
- e. Create array of objects using 1. Initializer list, 2. Dynamic initialization

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
class Book {
```

```
public:
```

```
    string title, author, ISBN;
```

```
    Book() {
```

```
        this->title = "";
```

```
        this->author = "";
```

```
        this->ISBN = "";
```

```
    }
```

```
    Book(string title, string author, string ISBN) {
```

```
        this->title = title;
```

```
        this->author = author;
```

```
        this->ISBN = ISBN;
```

```
    }
```

```
    Book(const Book &b) {
```

```
        this->title = b.title;
```

```
        this->author = b.author;
```

```
        this->ISBN = b.ISBN;
```

```
    }
```

```
};
```

```
class Library {
```

```
Book books[100];
```

```
int count;
```

```
public:
```

```
Library() { count = 0; }
```

```
bool addNewBook(string &title, string &author, string &ISBN) {
```

```
    if(count >= 100) return false;
```

```
    books[count] = Book(title, author, ISBN);
```

```
    count++;
```

```
    return true;
```

```
}
```

```
bool removeBooks(string &ISBN) {
```

```
    for(int i = 0; i < count; i++) {
```

```
        if(books[i].ISBN == ISBN) {
```

```
            for(int j = i; j < count - 1; j++) {
```

```
                books[j] = books[j + 1];
```

```
            }
```

```
            count--;
```

```
            return true;
```

```
        }
```

```
    }
```

```
    return false;
```

```
}
```

```
void displayDetails() {
```

```
    if(count == 0) {
```

```
        cout << "No books available\n";
```

```
        return;
```

```
    }
```

```
    for(int i = 0; i < count; i++) {
```

```

        cout << books[i].title << " | "
            << books[i].author << " | "
            << books[i].ISBN << endl;
    }
}
};

```

```

int main() {
    Book b1("C++", "Bjarne", "101"), b2("Java", "James", "102");
    Book b3 = b1;

```

```

    Book arr1[2] = {
        Book("DS", "Author1", "201"),
        Book("OS", "Author2", "202")
    };

```

```

    int n;
    cout << "Enter number of books: ";
    cin >> n;

```

```

    Book *arr2 = new Book[n];
    string t, a, i;

```

```

    for(int j = 0; j < n; j++) {
        cin >> t >> a >> i;
        arr2[j] = Book(t, a, i);
    }

```

```

    Library lib;
    int choice;

```

```

    do {
        cout << "1.Add 2.Remove 3.Display 4.Exit\n";

```

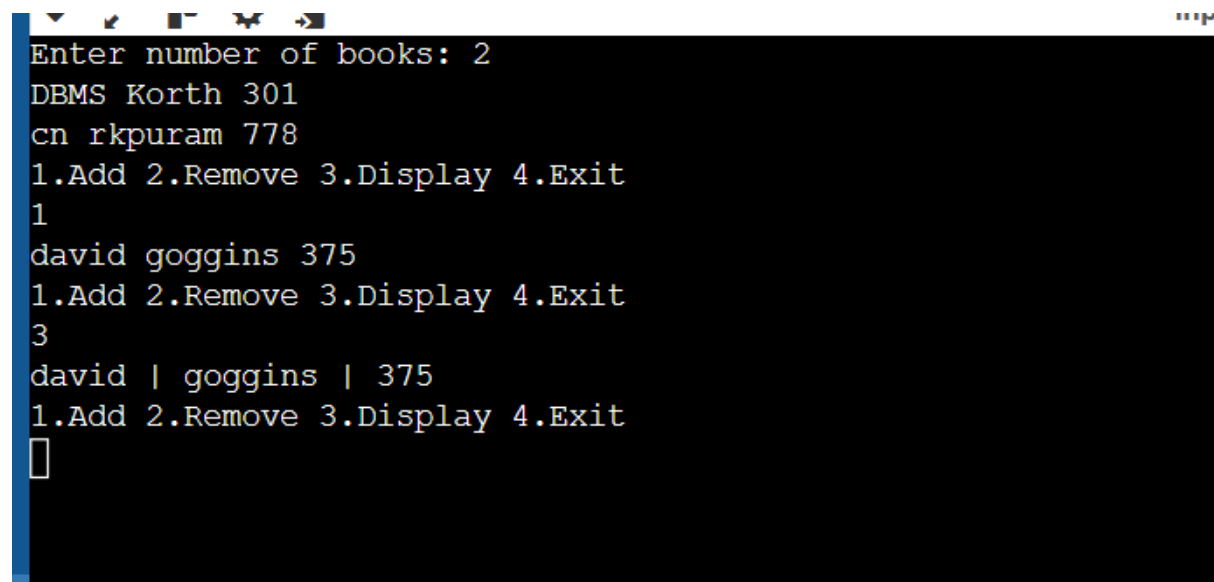
```

cin >> choice;

switch(choice) {
    case 1:
        cin >> t >> a >> i;
        lib.addNewBook(t, a, i);
        break;
    case 2:
        cin >> i;
        lib.removeBooks(i);
        break;
    case 3:
        lib.displayDetails();
        break;
}
} while(choice != 4);

delete[] arr2;
return 0;
}

```



```

Enter number of books: 2
DBMS Korth 301
cn rkpuram 778
1.Add 2.Remove 3.Display 4.Exit
1
david goggins 375
1.Add 2.Remove 3.Display 4.Exit
3
david | goggins | 375
1.Add 2.Remove 3.Display 4.Exit

```

3. Create a class *Account* with attributes *account number (of const long type)*, *transaction ID (of long type)*, *transaction type (credit/debit) (of string type)*, and *balance (of double type)*. The Account class should provide following methods:

- a. **long depositAmount(to, from, amount)**. The “to” and “from” are the account number of the sender and receiver. The function should return the transaction id.
- b. **long creditAmount(to, from, amount)** similar to the depositAmount function. This function should also return the transaction id.
- c. **void displayDetails()** should display all the details such as account number, remaining balance, transaction history (credited/debited). Make this function as the const function.
- d. In all the functions, make all the arguments as the const arguments.

Write a main function to demonstrate all the function calls. Create at least 5 accounts objects (using array or without array) to demonstrate the operations of credit and debit.

Note: Use the [scope resolution operator](#) to include in-built libraries (cin, cout, endl, string etc.)

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
class Account {
```

```
    const long accNo;
```

```
    long transactionID;
```

```
    string type;
```

```
    double balance;
```

```
    static long tid;
```

```
public:
```

```
    Account(long a, double b) : accNo(a) {
```

```
        balance = b;
```

```
        transactionID = 0;
```

```
        type = "None";
```

```
}
```

```
long depositAmount(const long to, const long from, const double amount) {
```

```
    if(accNo == to) {  
        balance += amount;  
        type = "Credit";  
        transactionID = ++tid;  
        return transactionID;  
    }
```

```
}
```

```
    if(accNo == from) {  
        balance -= amount;  
        type = "Debit";  
        transactionID = ++tid;  
        return transactionID;  
    }
```

```
}
```

```
    return -1;
```

```
}
```

```
long creditAmount(const long to, const long from, const double amount) {
```

```
    if(accNo == to) {  
        balance += amount;  
        type = "Credit";  
        transactionID = ++tid;  
        return transactionID;  
    }
```

```
}
```

```
    if(accNo == from) {  
        balance -= amount;  
        type = "Debit";  
        transactionID = ++tid;
```

```
        return transactionID;
    }
    return -1;
}
```

```
void displayDetails() const {
    cout << "Account No: " << accNo << endl;
    cout << "Balance: " << balance << endl;
    cout << "Last Transaction ID: " << transactionID << endl;
    cout << "Transaction Type: " << type << endl;
    cout << "-----\n";
}
};
```

```
long Account::tid = 1000;
```

```
int main() {
    Account a[5] = {
        Account(101, 1000),
        Account(102, 2000),
        Account(103, 3000),
        Account(104, 4000),
        Account(105, 5000)
    };
};
```

```
long from, to;
double amt;
```

```
cout << "Enter from account, to account and amount: ";
```

```

cin >> from >> to >> amt;

for(int i = 0; i < 5; i++) {
    a[i].depositAmount(to, from, amt);
}

for(int i = 0; i < 5; i++) {
    a[i].displayDetails();
}

return 0;
}

```

```

Enter from account, to account and amount: a b 7700
Account No: 101
Balance: 1000
Last Transaction ID: 0
Transaction Type: None
-----
Account No: 102
Balance: 2000
Last Transaction ID: 0
Transaction Type: None
-----
Account No: 103
Balance: 3000
Last Transaction ID: 0
Transaction Type: None
-----
Account No: 104
Balance: 4000
Last Transaction ID: 0
Transaction Type: None
-----
Account No: 105
Balance: 5000
Last Transaction ID: 0
Transaction Type: None
-----

...Program finished with exit code 0
Press ENTER to exit console.

```

4. Write a program to add data objects of two different classes using friend functions.



```
#include <iostream>
```

```
using namespace std;
```

```
class ClassB;
```

```
class ClassA {
```

```
    int a;
```

```
public:
```

```
    void input() {
```

```
        cin >> a;
```

```
    }
```

```
    friend int add(ClassA, ClassB);
```

```
};
```

```
class ClassB {
```

```
    int b;
```

```
public:
```

```
    void input() {
```

```
        cin >> b;
```

```
    }
```

```
    friend int add(ClassA, ClassB);
```

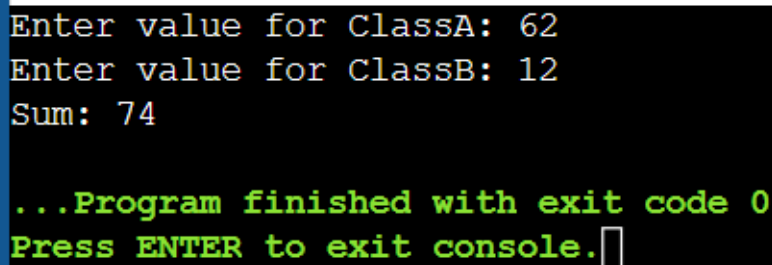
```
};
```

```
int add(ClassA x, ClassB y) {
```

```
    return x.a + y.b;
```

```
}
```

```
int main() {  
    ClassA obj1;  
    ClassB obj2;  
  
    cout << "Enter value for ClassA: ";  
    obj1.input();  
  
    cout << "Enter value for ClassB: ";  
    obj2.input();  
  
    cout << "Sum: " << add(obj1, obj2);  
  
    return 0;  
}
```



```
Enter value for ClassA: 62  
Enter value for ClassB: 12  
Sum: 74  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```

5. Define a class named **Complex** with properties (real and imaginary) and methods as per following details.
 - a. **Parameterized constructor** and **copy constructor** to initialize object values.
 - b. **void display ()** to display complex number.
 - c. **void sum (Complex, Complex)** to add two complex numbers (objects of Complex class). Implement this function as the friend function.

```
#include <iostream>
```

```
using namespace std;
```

```
class Complex {
```

```
    int real, imag;
```

```
public:
```

```
    Complex(int r = 0, int i = 0) {
```

```
        real = r;
```

```
        imag = i;
```

```
    }
```

```
    Complex(const Complex &c) {
```

```
        real = c.real;
```

```
        imag = c.imag;
```

```
    }
```

```
    void display() {
```

```
        cout << real << " + " << imag << "i" << endl;
```

```
    }
```

```
    friend void sum(Complex, Complex);
```

```
};
```

```
void sum(Complex c1, Complex c2) {
```

```
    Complex temp;
```

```
temp.real = c1.real + c2.real;

temp.imag = c1.imag + c2.imag;

temp.display();

}
```

```
int main() {

    int r1, i1, r2, i2;


    cout << "Enter real and imaginary part of first number: ";

    cin >> r1 >> i1;


    cout << "Enter real and imaginary part of second number: ";

    cin >> r2 >> i2;


    Complex c1(r1, i1);

    Complex c2(r2, i2);


    Complex c3 = c1;


    cout << "First number: ";

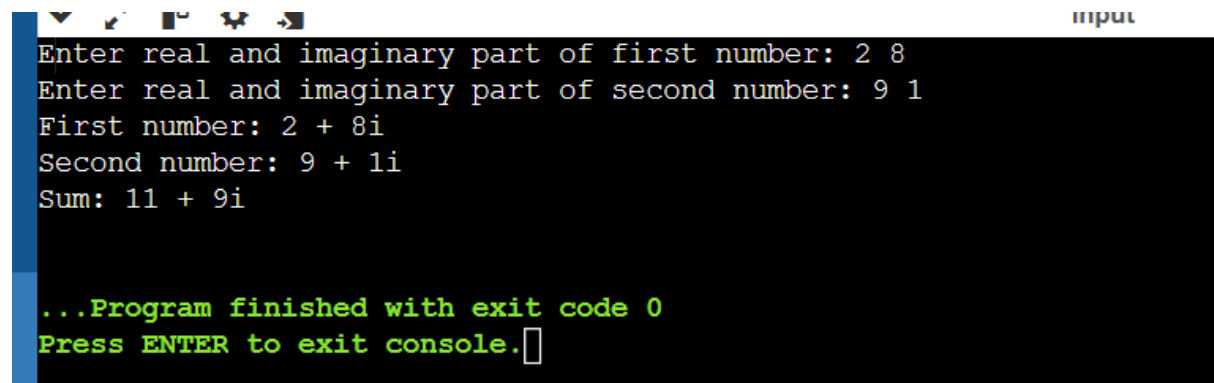
    c1.display();


    cout << "Second number: ";

    c2.display();

}
```

```
cout << "Sum: ";  
  
sum(c1, c2);  
  
return 0;  
  
}
```



The screenshot shows a terminal window with a dark background. At the top right, the word "input" is written in a light blue font. The terminal displays the following text in a monospaced font: "Enter real and imaginary part of first number: 2 8", "Enter real and imaginary part of second number: 9 1", "First number: 2 + 8i", "Second number: 9 + 1i", and "Sum: 11 + 9i". Below this, in green text, it says "...Program finished with exit code 0" and "Press ENTER to exit console." followed by a cursor icon. The terminal window has a standard Linux-style title bar with icons for window control on the left.

=====*****=====